
UNIT 12 QUERY PROCESSING AND EVALUATION

Structure	Page Nos.
12.0 Introduction	
12.1 Objectives	
12.2 Query Processing: An Introduction	
12.2.1 Role of Relational Algebra in Query Optimisation	
12.2.2 Using Statistics and Stored Size for Cost Estimation.	
12.3 Cost of Selection Operation	
12.3.1 File scan	
12.3.2 Index scan	
12.3.3 Implementation of Complex Selections	
12.4 Cost of Sorting	
12.5 Cost of Join Operation	
12.5.1 Block Nested-Loop Join	
12.5.2 Merge-Join	
12.5.3 Hash-Join	
12.6 Other Operations	
12.7 Representation and Evaluation of Query Expressions	
12.7.1 Evaluating a Query Tree.	
12.7.2 Evaluating Complex Joins	
12.8 Creation of Query Evaluation Plans	
12.8.1 Transformation of Relational Expressions	
12.8.2 Query Evaluation Plans	
12.8.3 Choosing an Optimal Evaluation Plan	
12.8.4 Cost and Storage-Based Query Optimisation	
12.9 View and Query Processing	
12.9.1 Materialised View	
12.9.2 Materialised Views and Query Optimisation	
12.10 Summary	
12.11 Solutions/Answers	

12.0 INTRODUCTION

The Query Language – SQL is one of the main reasons for the success of RDBMS. A user just needs to write the query in SQL that is close to the English language and does not need to say how such a query is to be evaluated. However, a query needs to be evaluated efficiently by the DBMS. But how is a query evaluated efficiently? This unit attempts to answer this question. The unit covers the basic principles of query evaluation, the cost of query evaluation, the evaluation of join queries, etc. in detail. It also provides information about query evaluation plans and the role of storage in query evaluation and optimisation. This unit introduces you to the complexity of query evaluation in DBMS.

12.1 OBJECTIVES

After going through this unit, you should be able to:

- Explain the measure of query cost;
- define algorithms for individual relational algebra operations;
- create and modify query expression;
- define evaluation plan choices, and
- define query processing using views.

12.2 QUERY PROCESSING: AN INTRODUCTION

Before defining the measures of query cost, let us begin by defining query processing. *Figure 1* shows the steps of query processing.

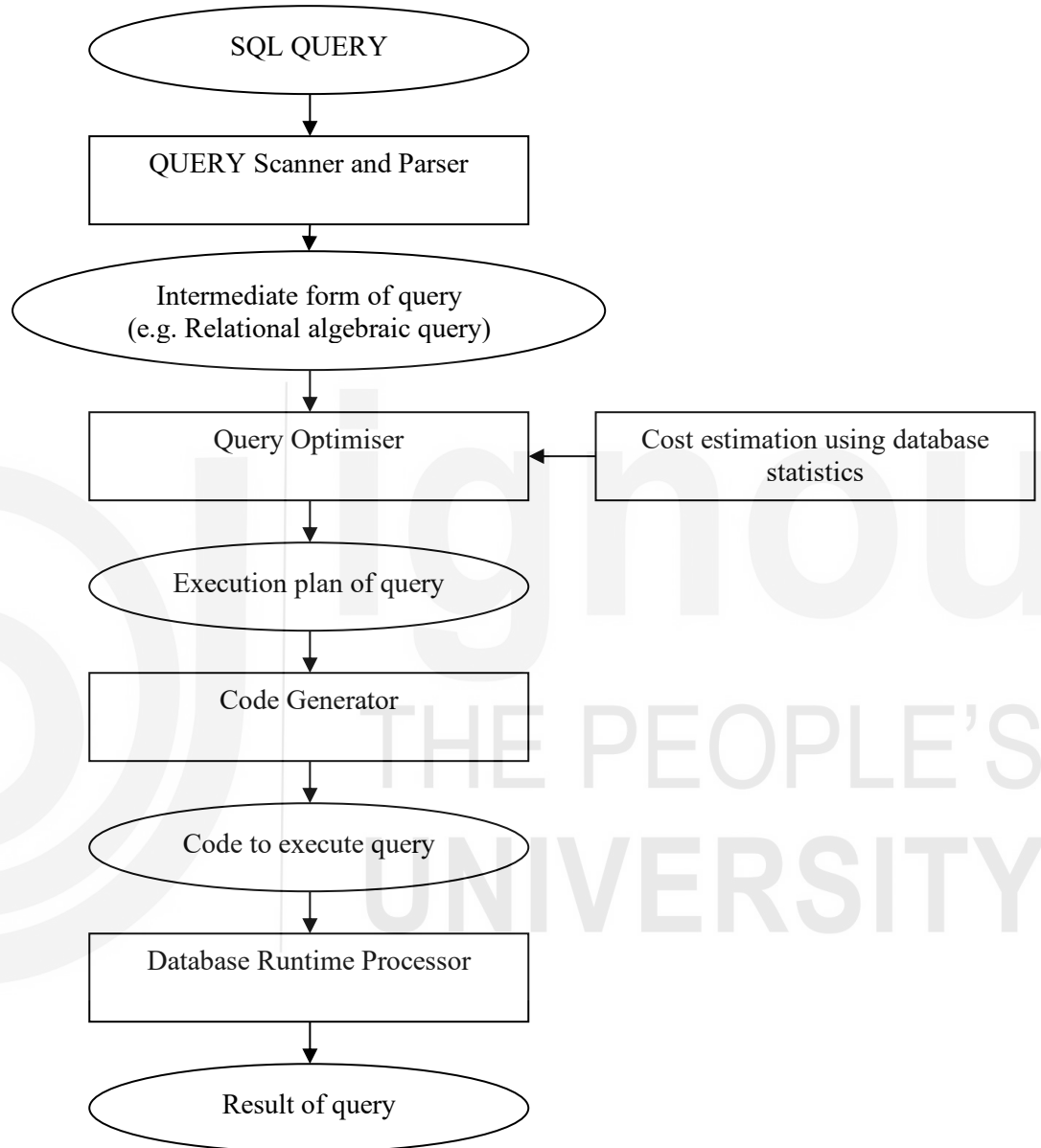


Figure 1: Query processing

In the first step, a query is scanned, parsed, validated and translated into an internal form. This internal form may be the relational algebra (an intermediate query form). The parser checks the syntax of the query and verifies the relations. Next, the query is optimised, and a query execution plan is generated, which is then compiled into a code that can be executed by the database runtime processor. The query processing involves the study of the following concepts:

- measures to find query cost.
- algorithms for evaluating relational algebraic operations.

- evaluating a complete expression using algorithms on individual operations.

12.2.1 Role of Relational Algebra in Query Optimisation

In order to optimise the evaluation of a query, first, you must define the query using relational algebra. A relational algebra expression may have many equivalent expressions. For example, the relational algebraic expression $\sigma_{(\text{salary} < 5000)}(\pi_{\text{salary}}(\text{EMP}))$ is equivalent to $\pi_{\text{salary}}(\sigma_{\text{salary} < 5000}(\text{EMP}))$. This may result in generating many alternative ways of evaluating the query.

Further, a relational algebraic expression can be evaluated in many different ways. A detailed evaluation strategy for an expression is known as an evaluation plan. For example, you can use an index on *salary* to find employees with *salary* < 5000, or you can perform a complete relation scan and discard employees with *salary* ≥ 5000. Both of these are separate evaluation plans. The basis of the selection of the best evaluation plan is the cost of these evaluation plans.

Query Optimisation: The query optimisation selects the query evaluation plan with the lowest cost among the equivalent query evaluation plans. Cost is estimated using the statistical information obtained from the database catalogue, viz., the number of tuples in each relation, the size of tuples, different values of an attribute, etc. The cost estimation is made on the basis of heuristic rules.

What is the basis for measuring the query cost? The next section addresses this question.

12.2.2 Using Statistics and Stored Size for Cost Estimation

The query cost is generally measured as the total elapsed time for answering the query. There are many factors that contribute to the cost in terms of elapsed time. These are the time of *disk accesses*, *CPU time*, and *data communication time on the network*. However, these times can be measured when the query is being executed. Therefore, you may use statistics, like the number of records, number of blocks, number of attributes, possible number of different values for each attribute, etc., to estimate the cost. However, disk access is typically the predominant cost as disk transfer is very slow. In addition, disk accesses are relatively easier to estimate. Therefore, the following disk access cost can be used to estimate the query cost:

Number of seeks × average-seek-time; and
 Number of blocks read × average-block-read-time; and
 Number of blocks written × average-block-write-time.

Please note that the cost of writing a block is higher than the cost of reading a block. This is due to the fact that the data is read back after being written to ensure that the write operation was successful. However, for the sake of simplicity, we will just use the *number of block transfers from the disk as the cost measure*. We will also ignore the difference in cost between sequential and random Input/Output, which **depends** on the search criteria, such as point/range query on an ordering field vs other fields; and the file structures: heap, sorted, hashed. In addition, the number of block transfer is also dependent on the use of indices, such as primary, clustering, secondary, B+ tree, etc. Other cost factors may include buffering, disk placement, view materialisation, overflow / free space management, etc. Please also note that CPU time and communication time are also not being considered for cost computation.

In the subsequent section, let us try to find the cost estimates of various operations. Please note that the stored size of a relation is an important cost estimate, if the entire database-related cost is to be estimated. The other statistics like the number of tuples or the number of attributes etc. are useful when only part of the database is to be considered for query processing.

12.3 COST OF SELECTION OPERATION

The selection operation can be performed in several ways. Let us discuss the algorithms and the related cost of performing selection operation.

12.3.1 File scan

File scan algorithms locate and retrieve records that fulfil a selection condition in a file. The following are the two basic file scan algorithms for selection operation:

- 1) *Linear search:* This algorithm scans each file block and tests all records to see whether their attributes match the selection condition.

The cost of this algorithm (in terms of block transfer): This algorithm would require reading all the blocks of the file, as it must test all the records for the specific condition.

$$\begin{aligned} \text{Cost}_{\text{To find records that match a given criteria}} &= \text{Size of database in terms of Number of blocks} \\ &= N_b. \end{aligned}$$

$$\begin{aligned} \text{Cost}_{\text{Finding a specific value of key attribute}} &= \text{Average number of block transfer for locating the value} \\ &\text{(on an average half of the file needs to be traversed) so the cost is} \\ &= N_b/2. \end{aligned}$$

Linear search can be applied regardless of selection condition or ordering of records in the file, or availability of indices.

- 2) *Binary search:* It is applicable when the selection is an equality comparison on an attribute on which the file is ordered. Assume that the blocks of a relation are stored continuously then, the cost can be estimated as:

$$\begin{aligned} \text{Cost}_{\text{binary search}} &= \text{Cost}_{\text{Block based Binary search to find first tuple that matches the search criteria}} \\ &+ \text{Cost}_{\text{fetching contiguous blocks till the records keep matching the criteria}} \\ &= \lceil \log_2 (N_b) \rceil + \frac{\text{Mean of the number of tuples with similar attribute values}}{\text{Size of a Block of data in terms of Number of tuples}} \end{aligned}$$

You may observe that the cost computation is based on the database statistics.

12.3.2 Index scan

The index scan can be used for cases where the database contains an index on an attribute set that forms the search key.

- 1) (a) *Scanning for equality condition on a Primary index:* These kinds of searches try to find a specific key value using the primary index of a database system. Since the search criteria include equality on the primary key, therefore, the output of this search would be just a single record or no record at all. The cost of the scan is defined as:

$$\text{Cost} = \text{The depth traversed in the index to locate the block pointer} + 1 \text{ (for transfer of block consisting of desired primary key value).}$$

-
- (b) *Hash key:* It retrieves a single block directly, thus, the cost in the hash key organisation is given as:
=Block transfer needed for finding hash target +1

- 2) *Primary index-scan for comparison*: Assuming that the relation is sorted on the attribute(s) that are being compared, ($<$, $>$ etc.), then we need to locate the first record satisfying the condition after which the records are scanned forward or backwards as the condition may be, displaying all the records. Thus, the cost, in this case, would be:

$$\text{Cost} = \text{Number of block transfers to locate the value in index} + \text{Transferring all the blocks of data satisfying that condition.}$$

Please note you can roughly compute the number of blocks satisfying the condition as:

$$\begin{aligned} &\text{Number of attribute values that satisfy the condition} \\ &\quad \times \text{average number of tuples per attribute value} \\ &\quad / \text{blocking factor of the relation.} \end{aligned}$$

- 3) (a) *Equality on search key of secondary index*: Retrieves a single record if the search key is a candidate key.

$$\text{Cost} = \text{cost of accessing index} + 1.$$

It retrieves multiple records if the search key is not a candidate key.

$$\text{Cost} = \text{cost of accessing index} + \text{number of records retrieved}$$

(It can be very expensive).

Each record may be on a different block, thus requiring one block access for each retrieved record. This is the worst-case cost.

(b) *Secondary index comparison*: For the queries of the type that use the comparison on *secondary index value* $>$ *value searched*, the index is used to find the first index entry which is greater than the *value searched*, thereafter, the indexed is scanned sequentially till the end, finding the pointers to records.

For the $\leq$ type query, just scan the leaf pages of the index to find the pointers to the records until the first entry that satisfies the condition is found. Thereafter, the index can be scanned till the records satisfy the condition to obtain pointers to the records.

You may please note that after you have found the pointers to the records using the index scan, retrieving those pointed records may require one block transfer for each record. Please note that linear file scans may be cheaper if many records are to be fetched.

12.3.3 Implementation of Complex Selections

Conjunction: Conjunction is basically a set of AND conditions.

Conjunctive selection using one index: In such case, select any algorithm given earlier on one or more conditions and then test remaining conditions on the selected tuples after fetching them into the memory buffer.

Conjunctive selection using the multiple-key index: Use appropriate composite (multiple-key) index if they are available.

Disjunction: Disjunctions are basically a set of OR conditions.

Disjunction using the union of identifiers is applicable if *all* conditions have available indices, otherwise, use linear scan. Use the corresponding index for

each condition, take the union of all the obtained sets of record pointers, and eliminate duplicates, then fetch data from the file.

Negation: Use linear scan on file. However, if very few records are available in the result and an index is applicable on an attribute, which is being negated, then find the satisfying records using the index and fetch them from the file.

12.4 COST OF SORTING

This section introduces the cost of query evaluation when it requires sorting of records. There are various methods that can be used in the following ways:

- 1) Use an existing applicable ordered index (e.g., B+ tree) to read the relation in sorted order.
- 2) Build an index on the relation, and then use the index to read the relation in sorted order. (Options 1 and 2 may lead to one block access per tuple).
- 3) Techniques like *quicksort* can be used for relations that fit in the memory.
- 4) *External sort-merge* is a good choice for relations that do not fit in the memory.

Once you decide on the sorting technique, you can find the cost of these algorithms to find the sorted file. You may refer to further readings for more details.

Check Your Progress 1

- 1) What are the basic steps in query processing?
.....
.....
- 2) How can the cost of a query be measured?
.....
.....
- 3) What are the various methods adopted for performing selection operation?
.....
.....

12.5 COST OF JOIN OPERATION

There are several algorithms that can be used to implement joins:

- Nested-loop join
- Block nested-loop join
- Indexed nested-loop join
- Merge-join
- Hash-join

The choice of join algorithm is based on the cost estimates. We will elaborate on only a few of these algorithms in this section. The following relations and related statistics will be used to elaborate those algorithms.

MARKS (enrollno, subjectcode, marks): 20000 rows, 500 blocks
STUDENT (enrollno, name, dob): 5000 rows, 200 blocks.

12.5.1 Block Nested-Loop Join

In this join approach, a complete block of the outer loop is joined with the complete block of the inner loop. We have chosen STUDENT as the outer relation, as it is smaller in size and, therefore, will result in a smaller number of overall block transfers.

The algorithm for this may be written as:

```
for each tuple  $s_i$  in block  $s$  of STUDENT
{
    for each tuple  $m_i$  in block  $m$  of MARKS
    {
        Check if  $s_i$  and  $m_i$  satisfy the join condition
        if they do output joined tuple to the result
    }
};
```

In the worst case, when only one block of both the relations can be stored in RAM, the estimation of block accesses is:

= Number of Blocks of outer relation (STUDENT) $\times$ Number of blocks of inner relation (MARKS) + Number of blocks of outer relation (STUDENT).

= $200 \times 500 + 200 = 100200$

In the best case, when the smaller relation can be completely in RAM, the number of block accesses would be = Blocks of STUDENT + Blocks of MARKS

= $200 + 500 = 700$

Improvements to Block Nested-Loop Algorithm

The following modifications improve the block Nested method:

- Use $M - 2$ disk blocks as the blocking unit for the outer relation, where M = memory size in terms of the number of blocks.
- Use one buffer block to buffer the inner relation.
- Use one buffer block to buffer the output.

This method minimises the number of iterations.

12.5.2 Merge-Join

The merge-join is applicable to equijoin and natural join operations only. It has the following process:

- 1) Sort both relations on the joining attribute (if not already sorted).
- 2) Merge the sorted relations to Join them. In this step, every pair with the same value on the joining attribute must be matched.

For example, consider the instances of STUDENT and MARKS relations, as given in Figure 2.

STUDENT			MARKS		
enrolno	Name	----	enrolno	subjectcode	Marks
1001	Ajay		1001	MCS-211	55
1002	Aman		1001	MCS-212	75
1005	Rakesh		1002	MCS-212	90
1100	Raman		1005	MCS-215	75

Block 1 of STUDENT relation

Block 1 of MARKS relation

Figure 2: Sample Relations for Computing Join

Computation of the number of block accesses:

Join operation on enrolment number would be as follows:

- i) The enrolment number 1001 of STUDENT relation will join with two tuples of MARKS relation as:

1001		1001	MCS-211
1001		1001	MCS-212

- ii) Joining on the key 1001 will be over as soon as you find 1002 in MARKS relations, as MARKS relation is also sorted on enrolno. Thus, the join will continue as Merge-Join operations and the following tuples will be output after the output of the first two tuples for 1001.

This will be followed by output.

1002		1002	MCS-212
1005		1005	MCS-215

You may observe that the cost of the Merge-Join operation can be computed based on the assumption that a block that is part of the merge is read only once. The basic underlying assumption is that the main memory is sufficiently large to accommodate all the tuples of a specific attribute join value. Therefore, the number of block accesses for Merge-Join is:

$$= \text{Blocks of STUDENT} + \text{Blocks of MARKS} + \text{the cost of sorting on enrolno (if relations are unsorted)}$$

12.5.3 Hash-Join

Hash-join can be performed for both the equijoin and natural join operations. A hash function h is applied on joining attributes to partition tuples of both relations. In the case of STUDENT and MARKS relations, the hash function h maps joining attribute (enrolno in our example case) values to $\{0, 1, \dots, n-1\}$.

The join attribute is hashed to the join-hash partitions. In the example of Figure 4, we have used the mod 5 function for hashing, therefore, $n = 05$.

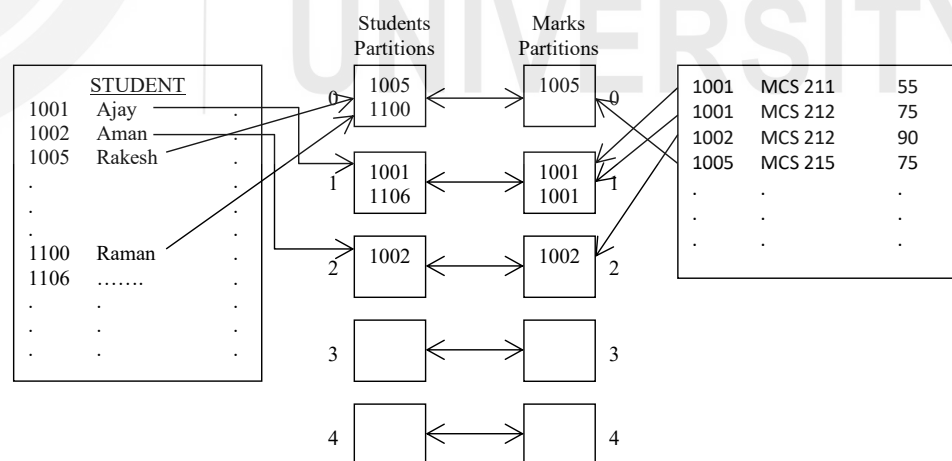


Figure 3: A hash-join example

Once the partition tables of STUDENT and MARKS are made on the enrolno, then only the corresponding partitions will participate in the join as:

A STUDENT tuple and a MARKS tuple that satisfy the Join condition will have the same value for the join attributes. Therefore, they will be hashed to an equivalent partition and, thus, can be joined easily. For example, STUDENT partition 1 consists of tuples of STUDENT with enrolno 1001, 1006; whereas

MARKS partition 1 consists of marks of the student enrolno 1001 in two different subjects. The join can now simply be performed for enrolno 1001. There is no joining MARKS tuple for STUDENT 1006.

Algorithm for Hash-Join

The hash-join of two relations r and s is computed as follows:

- Partition the relation r and s using the same hash function h . (When partitioning a relation, one block of memory is reserved as the output buffer for each partition). It may be noted that the tuples in i^{th} partition of r may join only with the tuples of i^{th} partition of s .
- For each partition s_i of s , load the partition into memory and build an in-memory hash index using the joining attribute. Why is this hash index needed? As several joining attributes may get mapped into the same partition. For example, partition 0 and partition 1 in Figure 3.
- Read the partition i of r into memory block by block. For each tuple in r_i , find the tuples s_i in the hash index of s_i , which would join with that tuple in r_i . Output the joined results.

The value n (the number of partitions) and the hash function h are chosen in such a manner that each s_i should fit into the memory. Typically, n is chosen as:

$$[\text{the Number of blocks of } s / \text{Number of memory buffers}] \times f$$

where f is typically around 1.2.

The partition of r can be large, as r_i need not fit in memory.

You may refer to the further readings for more details on hash join.

Cost calculation for Simple Hash-Join

- Cost of partitioning r and s :* all the blocks of r and s are read once and, after partitioning, written back to partitions, so
 $cost1 = 2 (\text{blocks of } r + \text{blocks of } s)$.
- The cost of performing the hash-join using the hash index on s will require at least one block transfer for reading the partitions.
 $cost2 = (\text{blocks of } r + \text{blocks of } s)$
- There are a few more blocks in the main memory that may be used for evaluation, they may be read or written back. We ignore this cost as it will be too low in comparison to $cost1$ and $cost2$.
 Thus, the total $cost = cost1 + cost2$
 $= 3 (\text{blocks of } r + \text{blocks of } s)$

Even if s is recursively partitioned, *hash-table overflow* can occur, i.e., some partition s_i may not fit in the memory. This may happen if many tuples in s have the same value for join attributes or the chosen hash function is bad. Partitioning is said to be *skewed* if some partitions have significantly more tuples than others. This is the overflow condition. The overflow can be handled in a variety of ways; however, they are beyond the scope of this unit.

Let us explain the hash join and its cost for the natural join $STUDENT \bowtie MARKS$. Assume a memory size of 25 blocks $\Rightarrow M=25$.

SELECT s as STUDENT as it has a smaller number of blocks (200 blocks) and r as MARKS (500 blocks).

$$\begin{aligned} \text{Number of partitions to be created for STUDENT} &= (\text{blocks of STUDENT} / M) \times 1.2 \\ &= (200 / 25) \times 1.2 = 9.6 \approx 10 \end{aligned}$$

Thus, the STUDENT relation will be partitioned into 10 partitions of 20 blocks each. MARKS will also have 10 partitions of 50 blocks each. The 25 buffers will be used as:
 20 blocks for one complete partition of STUDENT,
 01 block will be used for input of MARKS partitions, and
 The remaining 4 blocks may be used for storing the results.

The total cost = $3(200+500) = 2100$ as no recursive partitioning is needed.

12.6 OTHER OPERATIONS

There are many other operations that are performed in database systems. Let us introduce these operations in this section.

Duplicate Elimination: Duplicate elimination may be implemented by using hashing or sorting. On sorting, duplicates will be adjacent to each other thus, may be identified and deleted. An optimised method for duplicate elimination can be the deletion of duplicates during generation as well as at intermediate merge steps in an external sort-merge. Hashing is similar – duplicates will be clubbed together in the same bucket and, therefore, may be eliminated easily.

Projection: It may be implemented by performing the projection on each tuple, followed by duplicate elimination.

Aggregate Function Execution: Aggregate functions can be implemented in a manner similar to duplicate elimination. Sorting or hashing can be used to bring tuples in the same group together, and then aggregate functions can be applied to each group. For count, min, max, and sum, you may add up the aggregates. For calculating the average, take the sum of the aggregates and count the number of aggregates; and then divide the sum with the count at the end.

Set operations (such as $\cup$ and $\cap$) can either use a variant of merge-join after sorting or a variant of hash-join.

Using the Hashing:

- 1) Partition both relations using the same hash function, thereby creating partitions consisting of tuples of r and s , as:

$$r_0, r_1, \dots, r_{n-1} \text{ and } s_0, s_1, \dots, s_{n-1}$$

- 2) Process each partition i as follows:

Using a different hashing function, build an in-memory hash index on r_i after it is brought into the memory.

$r \cup s$: Add tuples in s_i to the hash index if they are not already in it. Output the hash index.

$r \cap s$: For each tuple s_i of the relation s , check if it is in the hash index, if yes, output this tuple.

$r - s$: For each tuple in s_i of the relation s , if the similar r_j is part of the hash index, delete r_j from the hash index. Once the entire s is processed, output the remaining hash index tuples.

There are many other operations as well. You may wish to refer to them in further readings.



Check Your Progress 2

- 1) Define the algorithm for Block Nested-Loop Join for the worst-case scenario.

.....

.....

- 2) What is the cost of Hash-Join?

.....

.....

- 3) What are the other operations that may be part of query evaluation?

.....

.....

12.7 REPRESENTATION AND EVALUATION OF QUERY EXPRESSIONS

Before we discuss the evaluation of a query expression, let us briefly explain how a SQL query may be represented. Consider the following STUDENT and MARKS relations:

STUDENT (enrolno, name, phone)

MARKS (enrolno, subjectcode, grade)

To find the result of the student(s) whose phone number is '1129250025', the following SQL query may be written:

```
SELECT enrolno, name, subjectcode, grade
FROM STUDENT s, MARKS m
WHERE s.enrolno=m.enrolno AND phone= '1129250025'
```

The equivalent relational algebraic query for this would be:

$$\pi_{\text{enrolno, name, subjectcode, grade}} ((\sigma_{\text{phone}='1129250025'} (\text{STUDENT}) \bowtie \text{MARKS}))$$

This is a very good internal representation; however, it may be a good idea to represent the relational algebraic expression as a query tree on which algorithms for query optimisation can be designed easily. In a query tree, nodes are the operators, and relations represent the leaf. The query tree for the relational expression above would be:

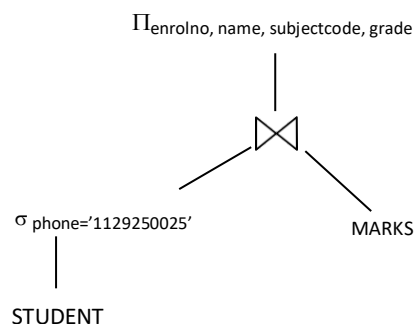


Figure 4: A Sample query tree

12.7.1 Evaluating a Query Tree.

In this section, let us examine the methods for evaluating a query expression that is expressed as a query tree. In general, we use two methods:

- Materialisation
- Pipelining.

Materialisation: Evaluates a relational algebraic expression in a bottom-up approach by explicitly generating and storing the results of each operation of expression. For example, for *Figure 4*, first selection operation on the STUDENT relation would be performed. Next, the result of the selection operation would be joined with the MARKS relation. Finally, the projection operation will be performed. Materialised evaluation is always possible even though the cost of writing/reading results to/from disk can be quite high.

Pipelining: Evaluates operations in a multi-threaded manner (i.e., passes tuples output from one operation to the next parent operation as input) even as the first operation is being executed. In the previous expression tree, it does not store (or materialise) results instead, it passes the tuples of the selection operation directly to the join operation. Similarly, it does not store the results of the join operation and passes the tuples of join operations directly to the projection operation. Thus, there is no need to store temporary relations on a disk for each operation. Pipelining may not always be possible or easy if sort or hash-join operations are used.

The pipelining execution method may involve a buffer, which is being filled by the result tuples of a lower-level operation, while records may be picked up from the buffer by a higher-level operation.

12.7.2 Evaluating Complex Joins

When an expression involves three relations, then you have more than one strategy for the evaluation of the expression. For example, join of relations such as STUDENT $\bowtie$ MARKS $\bowtie$ SUBJECTS may involve the following three strategies:

Strategy 1: Compute STUDENT $\bowtie$ MARKS, and *join* the result with SUBJECTS.

Strategy 2: Compute MARKS $\bowtie$ SUBJECTS first, and then *join* the result with STUDENT.

Strategy 3: Perform the pair of joins at the same time. This can be done by building an index of enrolno in STUDENT and on subjectcode in SUBJECTS. For each tuple *m* in MARKS, look up the corresponding tuples in STUDENT and the corresponding tuples in SUBJECTS. Each tuple of MARKS will be examined only once. Strategy 3 combines two operations into one special-purpose operation that may be more efficient than implementing the joins of two relations.

12.8 CREATION OF QUERY EVALUATION PLANS

We have already discussed query representation and its evaluation in the earlier section of this unit, but can something be done during these two stages that optimise the query evaluation? This section deals with this process in detail.

Generation of query-evaluation plans for a query expression involves several steps:

- 1) Generating logically equivalent expressions using **equivalence rules**
- 2) Generate alternative query plans.
- 3) Choose the cheapest plan based on the estimated cost.

The overall process is called cost-based **optimisation**. The cost difference between a good and a bad method of evaluating a query would be enormous. We need to estimate the cost of operations and statistical information about relations. For example, the number of tuples, the number of distinct values for an attribute, etc., helps estimate the size of the intermediate results, and information like available indices may help in estimating the cost of complex expressions. Let us discuss all the steps in query-evaluation plan development in more detail.

12.8.1 Transformation of Relational Expressions

Two relational algebraic expressions are said to be **equivalent** if, on every legal database instance, the two expressions generate the same set of tuples (the order of tuples is irrelevant). “Appendix-A”, given at the end of this unit, lists the rules that may be used to generate equivalent relational expressions. However, the rules given in Appendix A are too general, and a few heuristics rules generated from these rules can be used to transform the relational expressions. These rules are:

- (1) Combine a cascade of selections into a conjunction and test all the predicates on the tuples in a single iteration:
 Expression like: $\sigma_{\theta_2} (\sigma_{\theta_1} (E))$
 Can be converted to: $\sigma_{\theta_2 \wedge \theta_1} (E)$
- (2) Combining a cascade of projections into a single outer projection (please note that the final projection would be the outer projection).
 Expression like: $\pi_4 (\pi_3 (\dots(E)))$
 Can be converted to: $\pi_4 (E)$
- (3) Commutate the selection and projection or vice-versa. This commutation may sometimes reduce query cost.
- (4) Use associative or commutative rules for the Cartesian product or join operation to find various alternative paths for query evaluation.
- (5) Move the selection and projection (projection may be expanded to include join condition) before Join operations. The selection and projection result in the reduction of the number of tuples and, therefore, may reduce the cost of joining.
- (6) Commutate the projection and selection with Cartesian product or union.

Let us explain the use of some of these rules with the help of an example. Consider the query for the relations:

STUDENT (enrolno, name, phone)
 MARKS (enrolno, subjectcode, grade)
 SUBJECT (subjectcode, sname)

Example 1: Consider the query: Find the enrolment number, name, and grade of those students who have secured an A grade in the subject DBMS. One of the possible solutions to this query may be:

$\pi_{\text{enrolno, name, grade}} (\sigma_{(\text{sname} = \text{'DBMS'} \wedge \text{grade} = \text{'A'})} ((\text{STUDENT} \bowtie \text{MARKS}) \bowtie \text{SUBJECT}))$

The query tree for this would be:

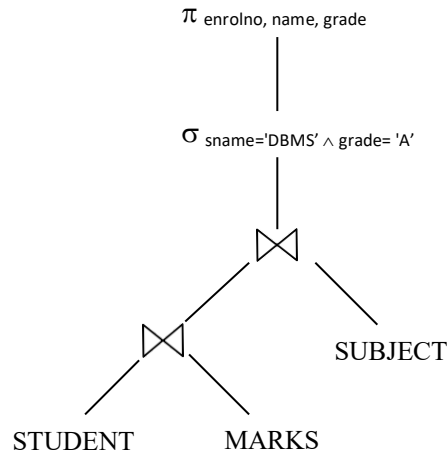


Figure 5: The query tree of example 1

As per the suggested rules, the selection condition may be moved before the join operation. The selection condition given in Figure 5 above is: $sname = 'DBMS'$ and $grade = 'A'$. Both of these conditions belong to different tables, as $sname$ is available only in the SUBJECT table and $grade$ in the MARKS table. Thus, the selection conditions will be mapped accordingly, as shown in Figure 6. Thus, the equivalent expression will be:

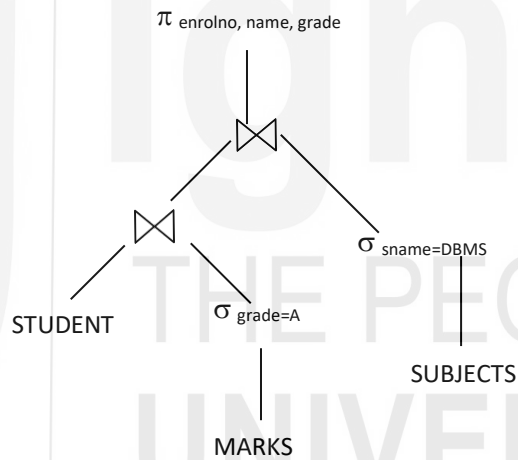


Figure 6: A modified tree

Further, the expected size of SUBJECT and MARKS after selection will be small, so it may be a good idea to join MARKS with SUBJECT first, and thereafter, the resultant relation is joined with the STUDENT relation. Hence, the associative law of JOIN may be applied.

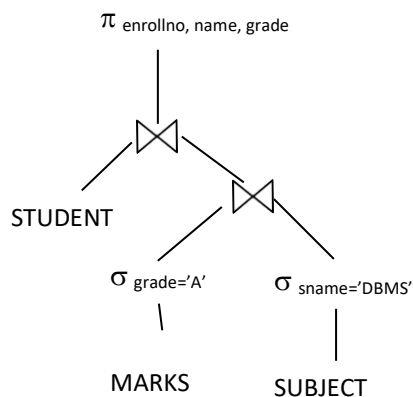


Figure 7: Modified query tree using associativity of join.

Even moving the projection before possible join, wherever possible, may optimise the query processing, as projection may also reduce the size of the intermediate result. Thus, you can move projection of outer join (refer to Figure 7) to inner join (Refer Figure 8).

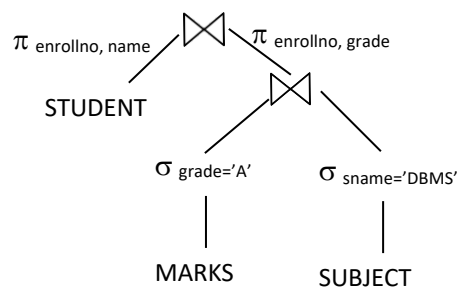


Figure 8: Moving the projection.

The final equivalent relation algebraic query expression of query of example 1 is:

$$(\pi_{\text{enrolno, name}}(\text{STUDENT})) \bowtie (\pi_{\text{enrolno, grade}}((\sigma_{\text{grade}='A'}(\text{MARKS})) \bowtie (\sigma_{\text{sname}='DBMS'}(\text{SUBJECT}))))$$

Alternative Query Expressions

A relational algebraic query can be optimised by the query optimiser, which creates several relational algebraic expressions that are equivalent to the query expression using the equivalence rules of relational algebra. One of the techniques to do so is to keep applying the equivalence rules to the relational algebraic query to generate a new set of expressions. However, this is a very time-consuming process. Therefore, in general, a set of heuristics, which are based on certain criteria like “apply those transformation rules that result in a reduction in the size of intermediate results”, can be used with the objective to produce more efficient equivalent query expressions.

12.8.2 Query Evaluation Plans

Let us first define the term Evaluation Plan. An evaluation plan defines exactly which algorithm is to be used for each operation and how the execution of the operation is coordinated. For example, Figure 9 shows the query tree with an evaluation plan.

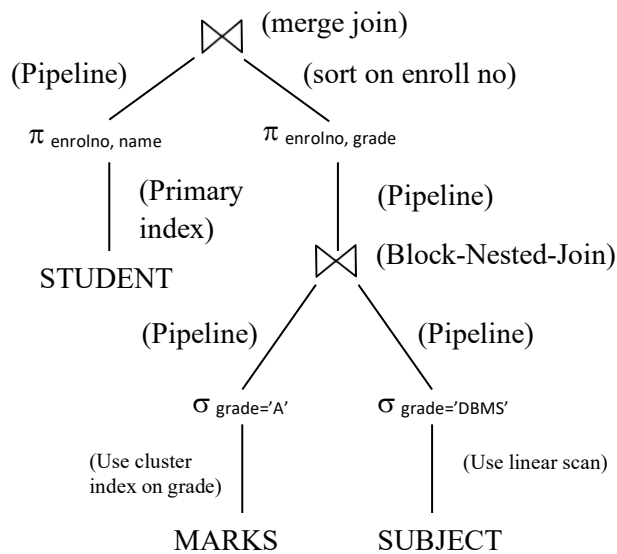


Figure 9: A sample query evaluation plan

12.8.3 Choosing an Optimal Evaluation Plan

The output of the previous step of query evaluation is the number of query evaluation plans. Which of these plans should be chosen for the final query evaluation? This decision is normally based on the database statistics. In addition, it is not necessary that individual best plans for each part of query evaluation may make the best query evaluation algorithm. For example, to join two relations, hash join is computationally less expensive than the merge join; however, the merge join produces sorted output. Thus, for the cases where sorted output may be useful for further processing of part results, merge join may be preferred. Similarly, the use of the nested loop method of joining may allow pipelining of the results of the join operation for further operations. In general, for choosing an optimal evaluation plan, you may perform cost-based optimisation to choose an optimal query evaluation plan.

12.8.4 Cost and Storage-Based Query Optimisation

Cost-based optimisation is performed on the basis of the cost of various individual operations that are to be performed as per the query evaluation plan. The cost is calculated as we have explained in section 12.3 with respect to the method and operation (JOIN, SELECT, etc.).

Storage and Query Optimisation

Cost calculations are primarily based on disk access; thus, storage has an important role to play in cost computation. In addition, some of the operations also require intermediate storage; thus, the cost is further enhanced in such cases. The cost of finding an optimal query plan is offset by the savings in terms of the query-execution time, particularly by reducing the number of slow disk accesses.

12.9 VIEWS AND QUERY PROCESSING

A view is defined as a query. The view may maintain the complete set of tuples following evaluation. This requires a lot of memory space; therefore, it may be a good idea to partially pre-evaluate it.

12.9.1 Materialised View

A materialised view is a view whose contents are computed and stored. Materialising the view would be very useful if the result of a view is required frequently, as it saves the effort of computing the view again.

Further, the task of keeping a materialised view up to date with the underlying data is known as materialised view maintenance. Materialised views can be maintained by re-computation on every update. A better option is to use incremental view maintenance, i.e., where only the affected part of the view is modified. View maintenance, in general, can be performed using triggers, which can be written for any data manipulation operation on the relations that are part of a view definition.

12.9.2 Materialised Views and Query Optimisation

We can perform query optimisation by rewriting queries to use materialised views. For example, assume that a materialised view of the join of two tables b and c is available as:

$$a = b \text{ NATURAL JOIN } c$$

Any query that uses natural join on b and c can use this materialised view ' a ' as:

Consider you are evaluating a query:

$$z = r \text{ NATURAL JOIN } b \text{ NATURAL JOIN } c$$

Then this query would be rewritten using the materialised view 'a' as:

$$z = r \text{ NATURAL JOIN } a$$

Do you need to perform materialisation? It depends on cost estimates for the two alternatives viz., use of a materialised view by view definition, or simple evaluation.

Query optimiser should be extended to consider all the alternatives of view evaluation and choose the best overall plan. This decision must be made on the basis of the system workload. Indices in such decision-making may be considered as specialised views. Some database systems provide tools to help the database administrator with index and materialised view selection.

Check Your Progress 3

- 1) List the methods used for the evaluation of expressions.

.....
.....

- 2) How do you define cost-based optimisation?

.....
.....
.....

- 3) Define the term "Evaluation plan".

.....
.....
.....

12.10 SUMMARY

This Unit introduces you to the basic concepts of query processing and evaluation. A query is to be represented in a standard form before it can be processed. In general, a query is evaluated after representing it using relational algebra. Thereafter, several query evaluation plans are generated using query transformations and an optimal plan is chosen for query evaluation. To find the cost of a query evaluation plan, you may use database statistics. The unit also defines various algorithms and the cost of these algorithms for evaluating various operations like select, project, join etc. You may refer to database textbooks for more details on query evaluation.

12.11 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) The steps of query evaluation are as follows:
 - a. In the first step, query scanning, parsing and validating is done.
 - b. Next, translate the query into a relational algebraic expression.
 - c. Next, the syntax is checked along with the names of the relations.

- d. Finally, the optimal query evaluation plan is executed and the
- e. answers to the query are returned.

- 2) Query cost is measured by considering the following activities:
- Number of disk-head seeks.
 - Number of blocks of tables.
 - Number of blocks of the results to be written

Please note that the cost measures above are based on the *number of disk blocks to memory buffer transfer*. We generally ignore the difference in cost between sequential and random I/O, CPU and communication costs.

- 3) The selection operation can be performed in several ways, such as:
Using File Scan: Using linear search or using the Binary search
Using Index Scan: Using the primary index (for equality) or the Hash key.
 You can also use a clustering index or a secondary index.

Check Your Progress 2

- 1) For each tuple in block B_r of r {
 For each tuple in block B_s of s {
 Test pair (t_i, s_i) to see if they satisfy the join condition
 If they do, add the joined tuple to the result.
 };
 };
- 2) The cost of Hash-join is.
 $3(\text{Blocks of } r + \text{blocks of } s)$
- 3) Some of the other operations in query evaluation are:
- Duplicate elimination
 - Projection
 - Aggregate functions.
 - Set operations.

Check Your Progress 3

- 1) Methods used for evaluation of expressions:
- (a) Materialisation
 - (b) Pipelining
- 2) Cost based optimisation consists of the following steps:
- (a) Generating logically equivalent expressions using equivalence rules
 - (b) Generate alternative query plans.
 - (c) Choose the cheapest plan based on the estimated cost.
- 3) The evaluation plan defines exactly what algorithms are to be used for each operation and the manner in which the operations are coordinated.

Equivalence Rules

- 1) The conjunctive selection operations can be equated to a sequence of individual selections. It can be represented as:

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- 2) The selection operations are commutative, that is,

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

- 3) Only the last of the sequence of projection operations is needed, the others can be omitted.

$$\pi_{\text{attriblist1}} (\pi_{\text{attriblist2}} (\pi_{\text{attriblist3}} \dots (E) ..) = \pi_{\text{attriblist1}} (E)$$

- 4) The selection operations can be combined with Cartesian products and theta join operations.

$$\sigma_{\theta_1} (E_1 \times E_2) = E_1 \bowtie_{\theta_1} E_2$$

and

$$\sigma_{\theta_2} (E_1 \bowtie_{\theta_1} E_2) = E_1 \bowtie_{\theta_2 \wedge \theta_1} E_2$$

- 5) The theta-join operations and natural joins are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

- 6) The Natural join operations are associative. Theta joins are also associative but with the proper distribution of joining conditions:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- 7) The selection operation distributes over the theta join operation under conditions when all the attributes in the selection predicate involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_1} (E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$$

- 8) The projections operation distributes over the theta join operation with only those attributes, which are present in that relation.

$$\pi_{\text{attriblist1} \cup \text{attriblist2}} (E_1 \bowtie_{\theta} E_2) = (\pi_{\text{attriblist1}} (E_1) \bowtie_{\theta} \pi_{\text{attriblist2}} (E_2))$$

- 9) The set operations of union and intersection are commutative. But set difference is not commutative.

$$E_1 \cup E_2 = E_2 \cup E_1 \text{ and similarly for the intersection.}$$

- 10) Set union and intersection operations are also associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3) \text{ and similarly for intersection.}$$

- 11) The selection operation can be distributed over the union, intersection, and set-differences operations.

$$\sigma_{\theta_1} (E_1 - E_2) = ((\sigma_{\theta_1} (E_1) - (\sigma_{\theta_1} (E_2)))$$

- 12) The projection operation can be distributed over the union.

$$\pi_{\text{attriblist1}} (E_1 \cup E_2) = \pi_{\text{attriblist1}} (E_1) \cup \pi_{\text{attriblist1}} (E_2)$$